src/chunking.py

Splitting documents into overlapping, citable chunks

Retrieval runs on **chunks**, not whole documents, because one embedding vector can only mean one thing. This module slices text into overlapping character windows and attaches each chunk's source metadata so answers can cite it.

Walkthrough of all 123 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Split documents into overlapping chunks for retrieval.
 2
 3  Why chunk at all? An embedding model turns a piece of text into ONE vector. Feed
 4  it a whole 4,000-word paper and that single vector has to average away every
 5  distinct idea in the paper — retrieval on it is mush. Feed it a few sentences and
 6  the vector actually means something specific, so a query about that specific idea
 7  lands on it. Chunking is what makes semantic retrieval precise.
 8
 9  The two knobs — chunk size and overlap — are named constants in `config.py` with
10  their rationale. This module is deliberately simple and character-based (no
11  tokenizer dependency) so the whole retrieval path stays reproducible and easy to
12  reason about; every chunk carries its parent paper's metadata so an answer can
13  cite the paper it came from.
14  """
```

Lines 1 to 14 are the module docstring. They justify chunking (a whole-paper embedding averages away every idea and retrieval on it is mush; a few sentences embed to something specific), note the two knobs live in config.py, and explain the deliberate choice of simple character-based splitting with no tokenizer dependency, plus the rule that every chunk carries its parent paper's metadata.

```
15
16  from __future__ import annotations
17
18  from dataclasses import dataclass
19
20  import pandas as pd
21
22  from . import config
23  from .acquisition import Paper
```

Lines 15 to 23 are imports: postponed annotations (line 16), dataclass for the Chunk type (line 18), pandas for the output frame (line 20), the project config (line 22), and the Paper type from acquisition (line 23) that this module consumes.

```
24
25
26  @dataclass
27  class Chunk:
28      """One retrievable unit of text plus the metadata needed to cite it."""
29
30      chunk_id: str        # e.g. "2401.01234:2" — paper id + position
31      arxiv_id: str
32      title: str
33      text: str
34      position: int        # 0-based index of this chunk within its paper
```

Lines 24 to 34 define the Chunk dataclass — one retrievable unit. Fields: chunk_id (line 30, formatted as paper-id colon position, e.g. 2401.01234:2), the arxiv_id and title for citation (lines 31 to 32), the chunk text (line 33), and position (line 34, the 0-based index of the chunk within its paper). The inline comments document the id format and the position meaning.

```
35
36
37  def split_text(text: str,
38                 size: int = config.CHUNK_SIZE_CHARS,
39                 overlap: int = config.CHUNK_OVERLAP_CHARS) -> list[str]:
40      """Slice `text` into overlapping windows of `size` characters.
41
42      Each window starts `size - overlap` characters after the previous one, so
43      consecutive chunks share `overlap` characters. That shared margin is what
44      stops a fact that straddles a boundary from being lost — it survives in at
45      least one chunk. We try to end each window at a whitespace boundary so chunks
46      don't split mid-word, which would corrupt the embedding.
47
48      Guarding the two edge cases explicitly: overlap must be smaller than size (or
49      the window never advances and we loop forever), and text shorter than one
50      window is returned as a single chunk.
51      """
52      if overlap >= size:
53          raise ValueError(f"overlap ({overlap}) must be smaller than size ({size})")
```

Lines 35 to 53 begin split_text. The signature (lines 37 to 39) defaults size and overlap to the config constants. The docstring (lines 40 to 51) explains the overlapping-window scheme and the two guarded edge cases. Lines 52 to 53 enforce the first guard: if overlap is greater than or equal to size the window would never advance, so it raises ValueError rather than looping forever.

```
54
55      text = text.strip()
56      if len(text) <= size:
57          return [text] if text else []
58
```

Lines 54 to 58 handle the short-text case. Line 55 strips surrounding whitespace. Lines 56 to 57 return the whole text as a single-element list when it fits in one window (or an empty list if the text was blank). Line 58 is a blank separator before the main loop.

```
59      chunks: list[str] = []
60      start = 0
61      while start < len(text):
62          end = min(start + size, len(text))
63          # Prefer to break at the last space in the window so we don't cut a word
64          # in half — but only if that space isn't so early it shrinks the chunk to
65          # nothing (which would happen on long unbroken strings like URLs).
66          if end < len(text):
67              last_space = text.rfind(" ", start, end)
68              if last_space > start + size // 2:
69                  end = last_space
70          chunks.append(text[start:end].strip())
71          if end >= len(text):
72              break
73          # Advance so the next window overlaps the current one by `overlap` chars.
74          start = end - overlap
75      return [c for c in chunks if c]
```

Lines 59 to 75 are the windowing loop. Line 59 initialises the chunks list; line 60 sets start to 0; line 61 loops while there is text left. Line 62 sets the window end. Lines 63 to 69 try to move that end back to the last space in the window (so words are not cut in half), but only if the space is past the window's midpoint, guarding against long unbroken strings like URLs. Line 70 appends the stripped window. Lines 71 to 72 break when the end reaches the text end. Line 74 advances start to end minus overlap, creating the intended overlap. Line 75 returns all non-empty chunks.

```
76
77
78  def chunk_paper(paper: Paper) -> list[Chunk]:
79      """Turn one paper into its list of Chunks, carrying metadata through."""
80      return [
81          Chunk(
82              chunk_id=f"{paper.arxiv_id}:{i}",
83              arxiv_id=paper.arxiv_id,
84              title=paper.title,
85              text=piece,
86              position=i,
87          )
88          for i, piece in enumerate(split_text(paper.text))
89      ]
```

Lines 76 to 89 define chunk_paper, which turns one Paper into a list of Chunks. It is a list comprehension (lines 80 to 88) that calls split_text on the paper's combined text and, for each resulting piece with its index i, builds a Chunk stamped with the paper's id (as id colon i), arxiv_id, title, the text piece, and position i — carrying the metadata through so citations survive.

```
90
91
92  def chunk_papers(papers: list[Paper]) -> list[Chunk]:
93      """Chunk every paper in the corpus."""
94      chunks: list[Chunk] = []
95      for p in papers:
96          chunks.extend(chunk_paper(p))
97      return chunks
```

Lines 90 to 97 define chunk_papers, which chunks the whole corpus. It initialises an empty list (line 94), loops over every paper (line 95), extends the list with that paper's chunks (line 96), and returns the flattened result (line 97).

```
 98
 99
100  def chunks_to_frame(chunks: list[Chunk]) -> pd.DataFrame:
101      """Chunks → DataFrame, the on-disk form and the shape the vector store wants."""
102      return pd.DataFrame(
103          {
104              "chunk_id": [c.chunk_id for c in chunks],
105              "arxiv_id": [c.arxiv_id for c in chunks],
106              "title": [c.title for c in chunks],
107              "text": [c.text for c in chunks],
108              "position": [c.position for c in chunks],
109          }
110      )
```

Lines 98 to 110 define chunks_to_frame, converting the list of Chunk objects into a pandas DataFrame — the on-disk form and the shape the vector store expects. It builds the frame column by column (lines 104 to 108), each column a list comprehension pulling one field from every chunk.

```
111
112
113  def save_chunks(chunks: list[Chunk]) -> pd.DataFrame:
114      """Chunk the corpus to parquet and return the frame. Called by the pipeline."""
115      config.ensure_dirs()
116      frame = chunks_to_frame(chunks)
117      frame.to_parquet(config.CHUNKS_FILE, index=False)
118      return frame
```

Lines 111 to 118 define save_chunks, called by the pipeline. It ensures the output directories exist (line 115), builds the frame (line 116), writes it to parquet without the row index (line 117), and returns the frame (line 118) so the caller can use it immediately without re-reading disk.

```
119
120
121  def load_chunks() -> pd.DataFrame:
122      """Load the chunk frame written by `save_chunks`."""
123      return pd.read_parquet(config.CHUNKS_FILE)
```

Lines 119 to 123 define load_chunks, the inverse: it reads and returns the chunk frame previously written by save_chunks from the configured parquet path.